

Two Stochastic Idiots Are Better Than One

A lightning talk about getting AI agents to supervise each other — and why it works.



Two AI agents working together produce better results than one working alone.

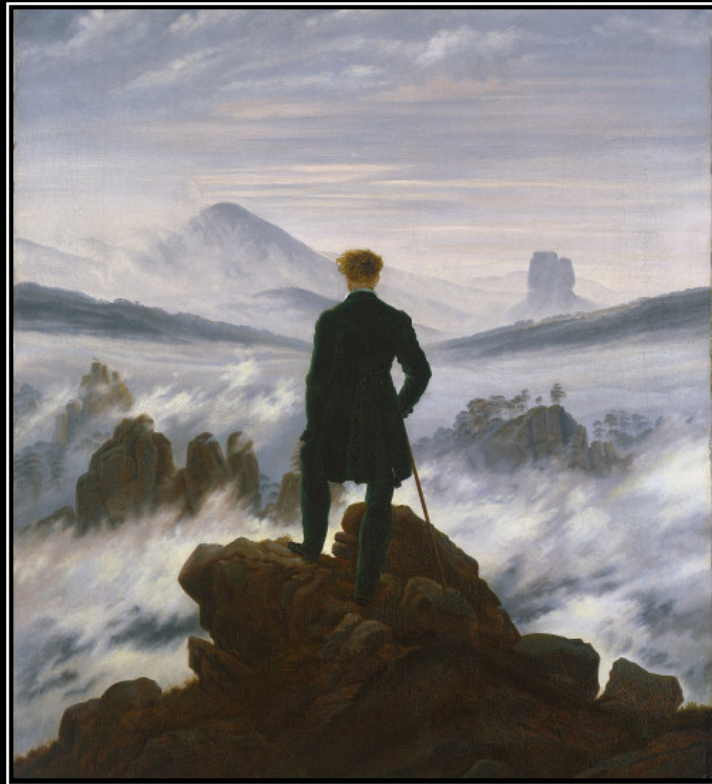
Not because they're smarter. Because they correct each other.

I'm not talking about anything clever. I'm talking about one agent writing code and another one nagging it to rebase.

And it works. Unreasonably well.

The Problem

"I should be more disciplined"



DISCIPLINE

A wish dressed as a plan

right

I was using an AI agent to write code. Good agent. Smart agent. Probably Opus according to my usage stats from James. But it could not remember to rebase/test/commit. Every single time — it forgot.

I asked the AI how to fix this. Its answer? "I should be more disciplined."

My process thinking hat went on. That's not a solution. That's a wish. Sure, there's no way to *guarantee* someone follows a rule correctly — [Wittgenstein][wittgenstein] will tell you that — but it doesn't mean we shouldn't try to make it *easier* to comply.

What I wanted was basically me, popping in every few minutes, shouting "REBASE. TEST. COMMIT."

Why Discipline Can't Work

It's not laziness. It's physics.



The instruction to "rebase, test, commit" goes in at the top of the context window. Then the agent starts working. Tool calls. Code. Errors. Retries. Output accumulates — thousands of tokens of it.

That early instruction doesn't disappear. It just gets drowned. The model attends to recent tokens far more than distant ones. Your behavioural rule is still technically *in* the context. It has no power.

Then the context fills up. Compaction kicks in. The agent summarises what came before — and a high-level instruction like "always rebase" is exactly the kind of thing that gets collapsed into nothing.

You're not dealing with a forgetful employee. You're dealing with an architecture that structurally cannot hold early instructions at full weight indefinitely. "Discipline" isn't a solution. It's a category error.

That said it *is* like dealing with a forgetful employee - or a forgetful *me*. Developers have often used techniques to keep them disciplined. For instance, if we want to keep line lengths short, we add a linter. If we want to commit regularly, we start a pomodoro timer (I don't - I'm even too lazy for that).

When I am trying to be disciplined, I add tools and techniques in to help me. That's what my robot buddy needs.

Finding hcom

Let AI agents message, wait, and spawn each other across terminals. Claude Code, Gemini CLI, Codex, OpenAI

agent cli automation
terminal ai communicat
opencode orchestration
multi-agent developer-tools
codex ai-agents claude
ai-tools gemini-cli
agent-skills claude-code
coding-agent subagents
claude-code-hooks

-  [Readme](#)
-  [MIT license](#)
-  [Activity](#)

That was what I needed. A second agent. A supervisor. A little pomodoro timer guy who would shout One whose entire job is to shout "REBASE. TEST. COMMIT."

So that's what I did. And it worked.

Why Does It Work?

First intuition — the dice



Roll one die. You get a 3. Or a 6. Or a 1. Wildly variable.

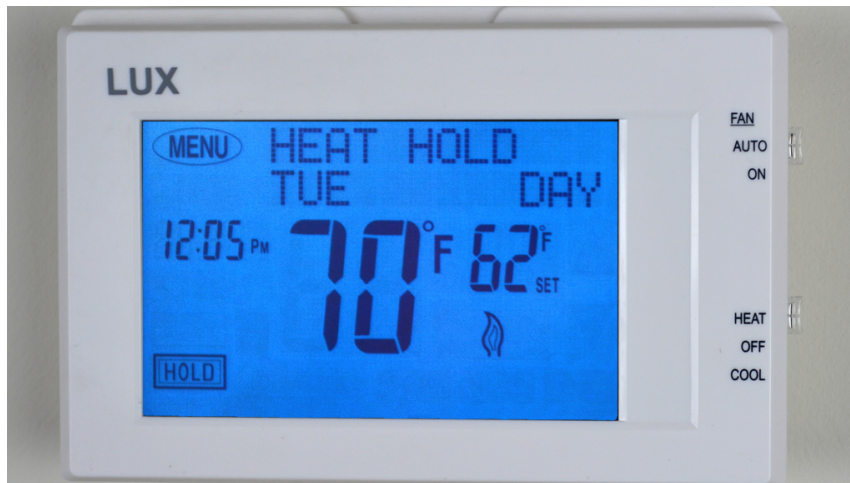
Roll a hundred dice, take the average. You get something close to 3.5. Every time.

[Central Limit Theorem][clt]. More samples, less variance. Simple enough.

But that's not actually what's happening here. The real mechanism is better.

Not Averaging — Correcting

The thermostat

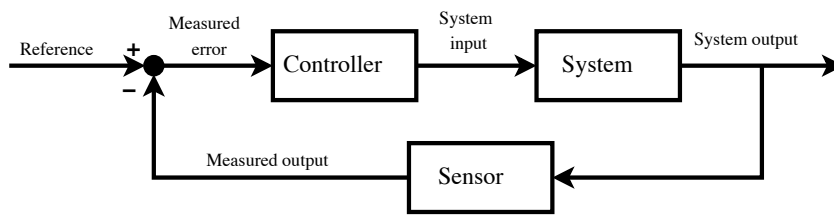


A thermostat doesn't average temperature readings and hope for the best. It *measures*, *compares* to a target, and *acts* to close the gap.

That's a closed-loop feedback system. Errors get corrected, not accumulated.

Two agents talking to each other *are* a feedback loop. The supervisor reads the worker's output. If it's drifting, it says so. The worker adjusts.

One agent alone is an open loop. No feedback. No correction. Just vibes.



The Actor-Critic Pattern

Discovered twenty-five years ago

Like all great patterns, this one already existed. [Konda and Tsitsiklis]
[konda-1999] formalised it in 1999:

- The **actor** decides what to do — the *policy*
- The **critic** evaluates how good that decision was — the *value function*

The actor proposes. The critic appraises. The actor learns from the appraisal.

Worker = actor. Supervisor = critic. We reinvented [reinforcement learning][wiki-ac] by accident.

Not Foolproof

Quis custodiet ipsos custodes?



Sometimes the supervisor gets off track. Forgets it's not meant to be *implementing*. Starts writing code instead of reviewing it.

Does it need its own supervisor? Infinite regress, each layer approaching but never reaching perfection...

For the most part though — it works. The supervisor stays on task. The worker stays disciplined. The code gets rebased, tested, and committed.

Needs to be codified. Turned into a repeatable skill. But the pattern is sound.



Two Idiots > One Genius

Neither agent is brilliant. The worker makes mistakes. The supervisor misses things. Individually, they're stochastic idiots. Sophisticated stochastic idiots. But still idiots.

But together — with a feedback loop between them — errors get caught, discipline holds, and the output converges on something better than either could produce alone.

Not because they're averaging out noise. Because they're *correcting* each other.

Two stochastic idiots. One closed loop. Zero babysitting.

The Industry Noticed

Frameworks built on the same pattern

Nobody sat down and derived actor-critic from first principles. They just noticed that multi-agent systems worked better — and built tooling around it.

- [AutoGen](#) (Microsoft)
- [LangGraph](#)
- [CrewAI](#)
- [OpenAI's Swarm](#)

All of them have explicit supervisor/worker patterns. The pattern is everywhere.

Questions?